

COS214 Prac 4

Caleb Jennings - u25173805

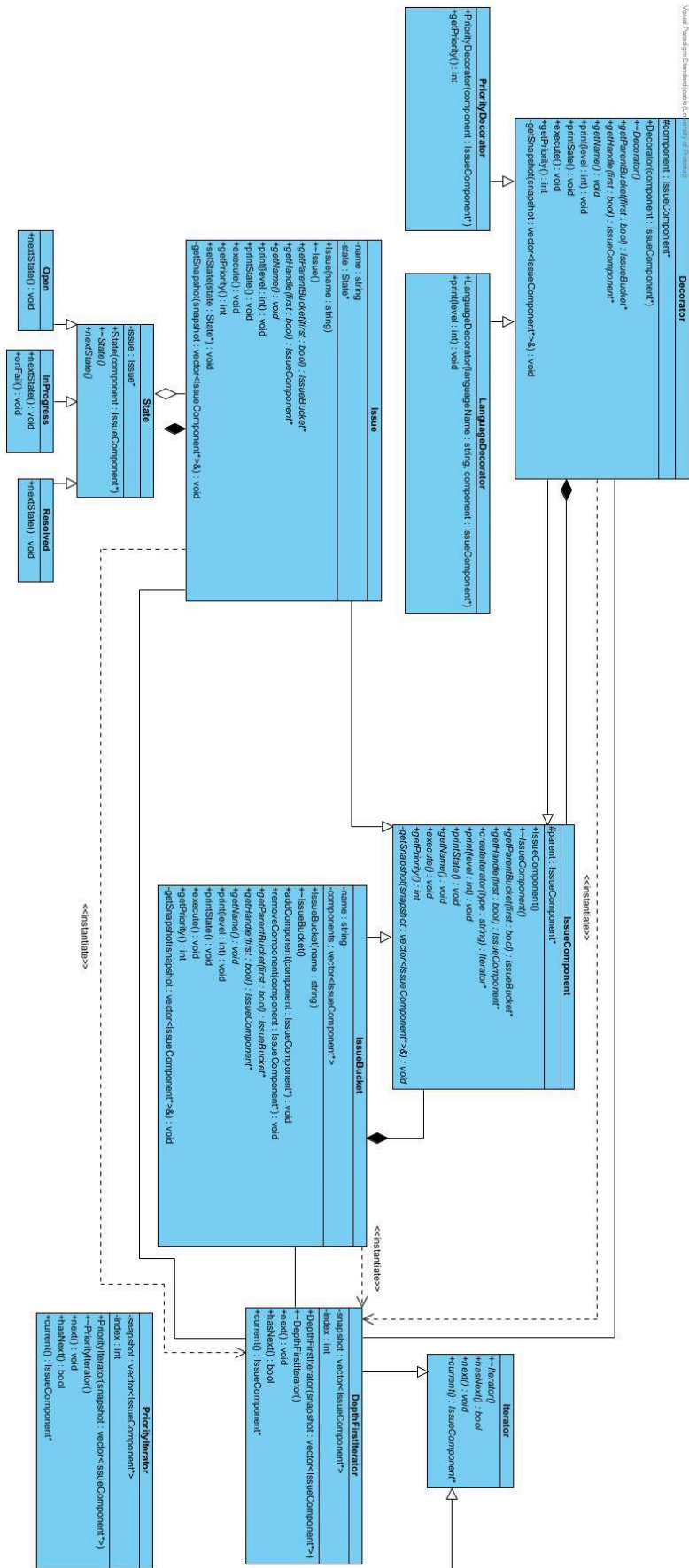
Corné Myburgh - u25505204

Reneilwe Molopyane - u25161874

System Design

TaskForge is a task-tracking and project management platform designed to organize, structure, and monitor features, bugs and changes. Modern software development requires implementations on different granularity. Some are individual bug fixes and others are features that require a number of tasks to be implemented in a complex hierarchy. TaskForge solves this problem by providing a structural framework with the “Composite” design pattern. It encapsulates this structure with a snapshot Iterator design that solves the issue of exposing the internal structure of the Composite. TaskForge has single tasks called “*Issues*”. These tasks can be grouped together by placing them in an “*IssueBucket*”. *IssueBuckets* can store many *issues* as well as other *IssueBuckets*. The *issues* are atomic tasks. *Issues* have states that change based on the execution. *Issues* can either be *Open*, *InProgress* or *Resolved*. Each *Issue* or *IssueBucket* can be decorated with the Languages it uses or the Priority of the *Issue*. There are two ways to go through the list of *issues*. Firstly, by going through every *issue* depth first. Secondly, by going through the high priority *issues*.

Class Diagram



GoF Participant Mapping

Composite

Component: IssueComponent

Composite: IssueBucket

Leaf: Issue

State

Context: Issue

State: State

ConcreteState: Open, InProgress, Resolved

Decorator

Component: IssueComponent

ConcreteComponent: Issue, IssueBucket

Decorator: Decorator

ConcreteDecorator: PriorityDecorator, LanguageDecorator

Iterator

Aggregate: IssueComponent

ConcreteAggregate: Issue, IssueBucket, Decorator

Iterator: Iterator

ConcreteIterator: DepthFirstIterator, PriorityIterator

Ownership Rationale

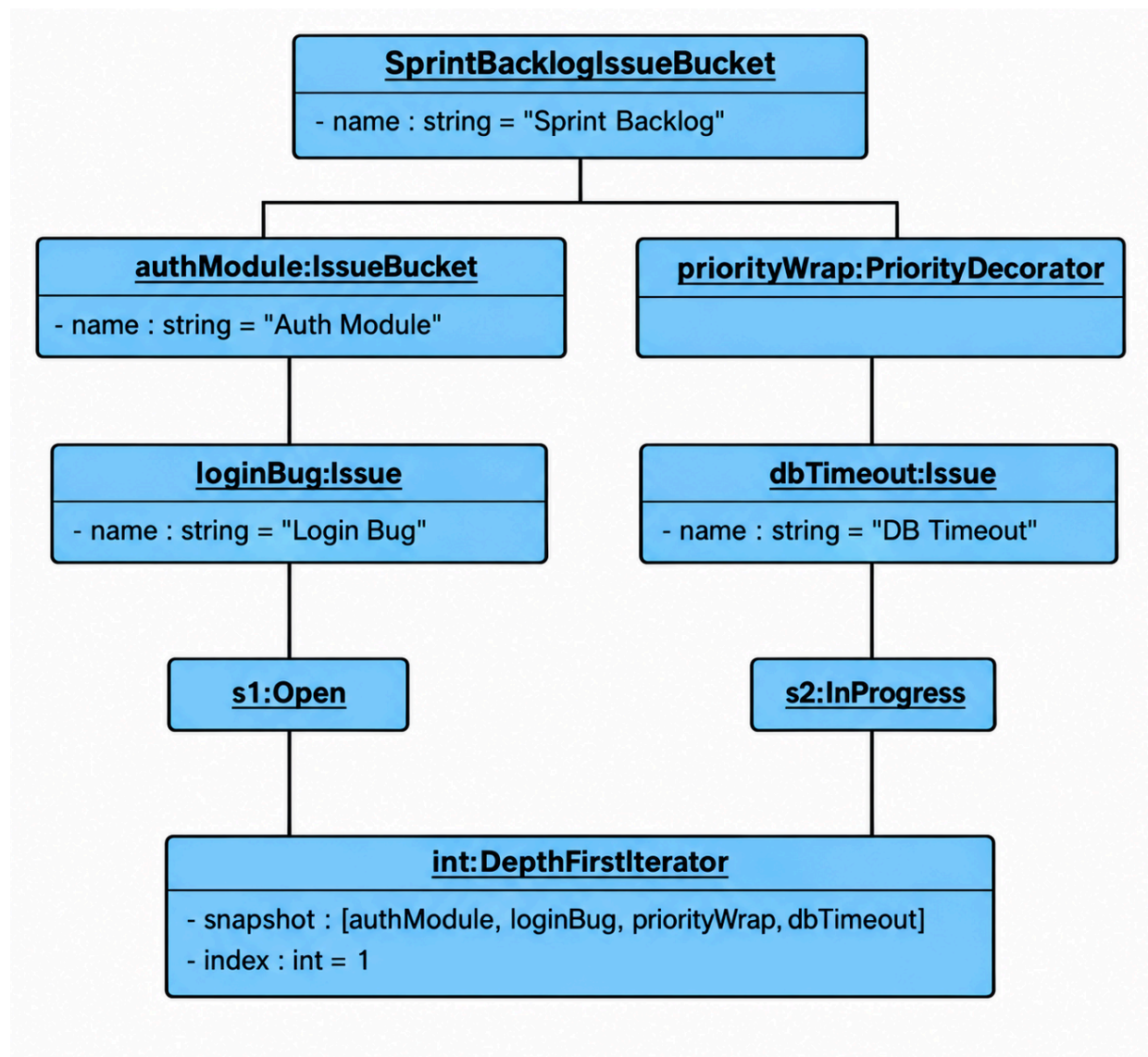
Relating to composite: An *IssueBucket* owns its children *IssueComponent*'s and will clean them up upon deletion. Similarly, a decorator wrapping a component will clean up the wrapped component.

When *createIterator()* is called on a component, the component does not own the iterator. Instead, the context that called *createIterator()* is responsible for cleaning it up after it has been used.

Issue's are the only component type with a state, and an issue owns its own state. When *setState()* is called on an issue, it will clean up its old state before setting the new one which is now owned by the issue.

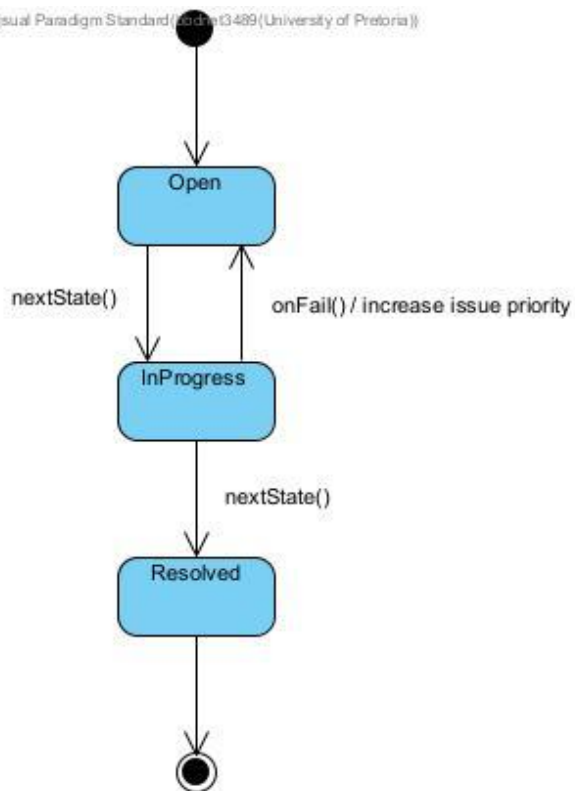
We set up the system this way because it is logical. Each component owned by a context is generally only used by that context.

Object Diagram



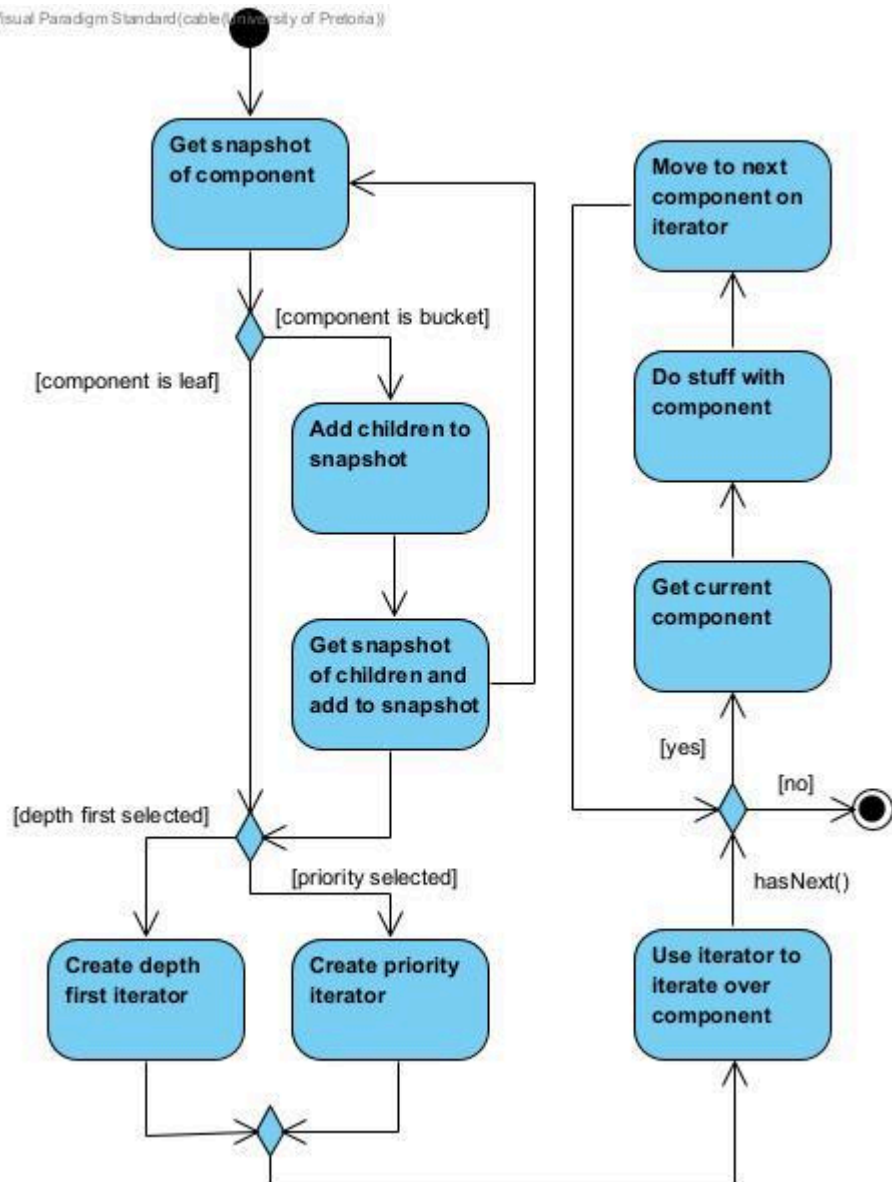
State Diagram

Visual Paradigm Standard (code: 3489 (University of Pretoria))

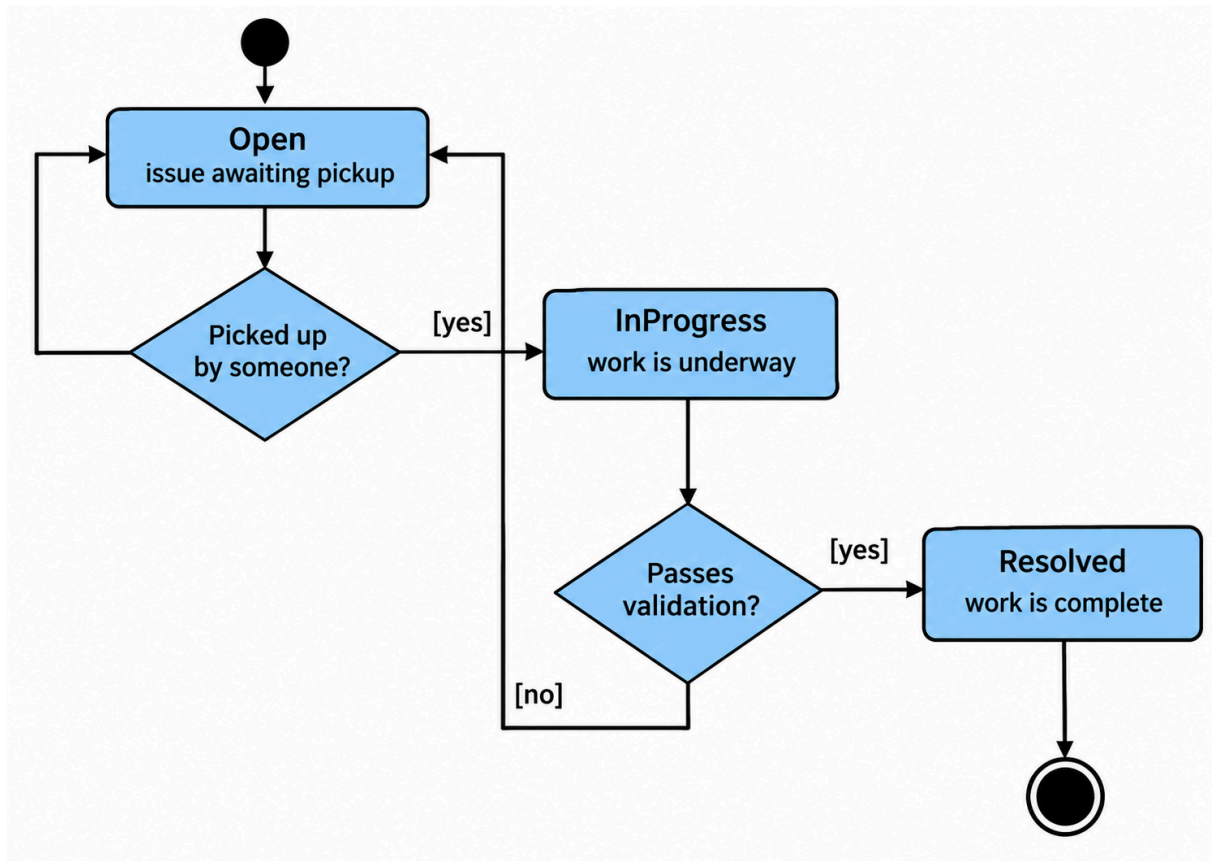


Activity Diagrams

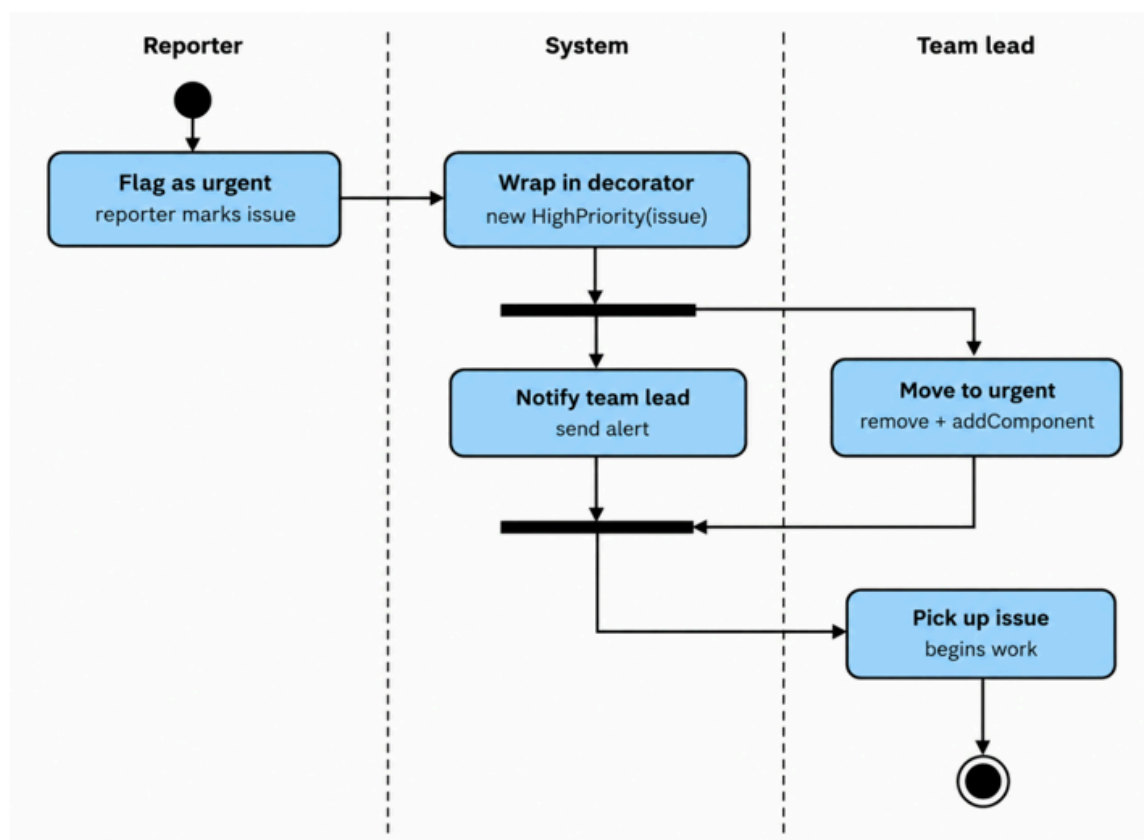
1. Traversal Activity Diagram



2. State transitions of an issue Activity Diagram



3. Priority Escalation Activity Diagram



Traversal-Modification Policy

We decided to use the snapshot Traversal-Modification policy. When `createIterator` is called on a component it creates an immediate snapshot of itself and all of its children. This iterator then holds a pointer to all of the necessary components it needs for traversal.

Policy for changes at runtime:

The iterators do not break when changes are made because they store their own local vector. Iterators traverse their own isolated vector of pointers so structural changes won't affect their current position.

Modification of the existing components will reflect on every created iterator as they store the pointer to those components. This means if an issue gets resolved on one iterator the other iterators will have this change as well.

Drawback: Iterators that use a snapshot can become outdated. Because newly added children or decorators will not be reflected in old iterators. Only the new iterators will have this change, **BUT** luckily this does not break old iterators and does not have that great of an effect. The old iterators will simply just finish going through their pointers.

Performance & Collaboration

GDB Bug Investigation & Valgrind

GDB:

```
(gdb) break main.cpp:92
Breakpoint 1 at 0x4248: file main.cpp, line 92.
(gdb) break PriorityIterator::hasNext
Breakpoint 2 at 0x8b73: file src/iterator/PriorityIterator.cpp, line 13.
(gdb) run
Starting program:
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge
```

This GDB supports auto-downloading debuginfo from the following URLs:

```
<https://debuginfod.ubuntu.com>
```

Enable debuginfod for this session? (y or [n]) n

Debuginfod has been disabled.

To make this setting permanent, add 'set debuginfod enabled off' to `.gdbinit`.

[Thread debugging using libthread_db enabled]

Using host libthread_db library `"/lib/x86_64-linux-gnu/libthread_db.so.1"`.

```
=== Scenario 1 ===
```

```
...
```

```
=== Scenario 1 END ===
```

```
=== Scenario 2 ===
```

You've been assigned to a new game development project!
With a tight milestone coming up, you can't just fix issues in arbitrary order.
You need to focus on going through high-priority tasks first to unblock the team.

Breakpoint 1, main () at main.cpp:92

```
92      IssueBucket* gameFunctionality = new IssueBucket("gameFunctionality");
(gdb) print game
$1 = (IssueBucket *) 0x55555557ec30
(gdb) print *game
$2 = {<IssueComponent> = {_vptr.IssueComponent = 0x555555568968 <vtable for
IssueBucket+16>, parent = 0x0}, name = "GameCreation",
  components = std::vector of length 2, capacity 2 = {0x55555557e2b0, 0x55555557e350}}
(gdb) print PriorityIterator
$3 = (Iterator *) 0x7ffff78aab06 <__glibc_morecore+22>
(gdb) continue
Continuing.
Creating PriorityIterator...
Priority Iterator created
```

Iterating through PriorityIterator...

Breakpoint 2, PriorityIterator::hasNext (this=0x55555557e650) at
src/iterator/PriorityIterator.cpp:13

```
13      if (this->index >= static_cast<int>(this->snapshot.size())) {
(gdb) next
15      } else return true;
(gdb) print this->index
$4 = 0
(gdb) print this->snapshot[this->index]->getName()
$5 = "StartMenu"
(gdb) print this->snapshot[this->index]->getPriority()
$6 = 2
(gdb) finish
```

Run till exit from #0 PriorityIterator::hasNext (this=0x55555557e650) at
src/iterator/PriorityIterator.cpp:15

0x00005555555588ed in main () at main.cpp:110

```
110      while (PriorityIterator->hasNext()) {
```

Value returned is \$7 = true

```
(gdb) delete 2
```

```
(gdb) continue
```

Continuing.

We got an issue. Let's work on it!

Executing issue: StartMenu

Check its state now!

Issue: StartMenu [In progress]

Let's try to complete it

Executing issue: StartMenu
Success! Issue is now Resolved

If we messed up it will be set back to open, else it will be resolved
Now let's check the status
Issue: StartMenu [Resolved]

DFS Iterator moved to next component
Current: PlayerMovement

We got an issue. Let's work on it!
Executing issue: PlayerMovement
Check its state now!
Issue: PlayerMovement [In progress]
With language: C#

Let's try to complete it
Executing issue: PlayerMovement
Success! Issue is now Resolved

If we messed up it will be set back to open, else it will be resolved
Now let's check the status
Issue: PlayerMovement [Resolved]
With language: C#

DFS Iterator reached the end of the traversal.

Structure after Scenario2
Bucket: GameCreation
 Bucket: Graphics
 Issue: CharacterModel [Open]
 Issue: World [Open]
 Bucket: Engine
 Bucket: gameFunctionality
 Issue: StartMenu [Resolved]
 Issue: PlayerMovement [Resolved]
 With language: C#

=== Scenario 2 END ===

[Inferior 1 (process 20567) exited normally]
(gdb) q

Bug:
./taskforge
Bucket: WebsiteCreation

Bucket: FrontEnd
*make: *** [Makefile:16: all] Segmentation fault (core dumped)*

After further investigation the error/bug comes from:

When creating an Issue its state was not automatically set. This had to be done through main, but the design of our relationships and visibility did not support a good way to do it from main. This also meant that the client had to do the heavy lifting for initialising every Issue's state. When calling print on the Issue it wanted to print the state it was in, but state was never set here.

Solution:

Issue initialises its State in the constructor.

Some changes outside Issue that made this possible for Decorated Issues to have correctly initialised State:

IssueComponent got variable (parent: IssueComponent)

The parent points to either a decorator or bucket.

Then there's a getParentBucket method in IssueComponent that returns that component's bucket and getHandle() inside IssueComponent returns the top-level decorator of a component.

After solution (No error):

./taskforge

Bucket: WebsiteCreation

Bucket: FrontEnd

Issue: Buttons [Open]

Issue: Headings [Open]

Bucket: Functionality

Issue: Functions [Open]

Bucket: BackEnd

Issue: Identity_Authentication [Open]

With language: JavaScript

Valgrind:

==8993==

==8993== HEAP SUMMARY:

==8993== in use at exit: 239 bytes in 5 blocks

==8993== total heap usage: 32 allocs, 27 frees, 75,975 bytes allocated

==8993==

==8993== 31 bytes in 1 blocks are indirectly lost in loss record 1 of 5

==8993== at 0x4846FA3: operator new(unsigned long) (in

/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)

==8993== by 0x49C863F: std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >::_M_assign(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)

==8993== by 0x49C8B24: std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >::operator=(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)

```

==8993== by 0x10E069: Issue::Issue(std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> >) (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993== by 0x10A885: main (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993==
==8993== 48 bytes in 1 blocks are indirectly lost in loss record 2 of 5
==8993== at 0x4846FA3: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8993== by 0x10E073: Issue::Issue(std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> >) (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993== by 0x10A70B: main (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993==
==8993== 48 bytes in 1 blocks are indirectly lost in loss record 3 of 5
==8993== at 0x4846FA3: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8993== by 0x10E073: Issue::Issue(std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> >) (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993== by 0x10A885: main (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993==
==8993== 104 (56 direct, 48 indirect) bytes in 1 blocks are definitely lost in loss record 4 of 5
==8993== at 0x4846FA3: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8993== by 0x10A6C6: main (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993==
==8993== 135 (56 direct, 79 indirect) bytes in 1 blocks are definitely lost in loss record 5 of 5
==8993== at 0x4846FA3: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8993== by 0x10A840: main (in
/home/coco/HOMEWORK/COS214_Pracs/Prac4/COS214-Prac4/taskforge)
==8993==
==8993== LEAK SUMMARY:
==8993== definitely lost: 112 bytes in 2 blocks
==8993== indirectly lost: 127 bytes in 3 blocks
==8993== possibly lost: 0 bytes in 0 blocks
==8993== still reachable: 0 bytes in 0 blocks
==8993== suppressed: 0 bytes in 0 blocks
==8993==
==8993== For lists of detected and suppressed errors, rerun with: -s
==8993== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)

```

Leak came from Decorator not correctly deleting its wrapped component. This leak was quite hidden as we thought it came from Composite. After fixing Decorator destructor:

```
==9813==  
==9813== HEAP SUMMARY:  
==9813==    in use at exit: 0 bytes in 0 blocks  
==9813== total heap usage: 32 allocs, 32 frees, 75,975 bytes allocated  
==9813==  
==9813== All heap blocks were freed -- no leaks are possible  
==9813==  
==9813== For lists of detected and suppressed errors, rerun with: -s  
==9813== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

GitHub

[Repo Link](#)

We used GitHub for collaboration as follows: Whenever one of us wanted to work on a new feature, we created a new branch locally, made a few commits with the new changes, and then pushed the branch to the repo. Once on the repo, the person who made those changes opened a pull request for the branch and requested review from one or both of the other members. If the changes were not approved, the pull request was closed and the person making the changes had to revise their additions. Otherwise, the changes are merged into main and the feature branch gets auto closed. The repository was set up in such a way that prevented pushing straight to main. Therefore, we all had to contribute to the main branch through pull requests.

Branches and pull requests were very helpful because they let us review each other's code before merging it into main, meaning all of our code on the main branch is agreed to be quality code.

Team Contribution Statement

Corné kicked things off by doing the initial design, the full class diagram and figuring out where each of the four GoF patterns fit ,with Reneilwe and Caleb chipping in ideas along the way. After that, implementation got split up by pattern: Reneilwe took Composite [commit 1d7e08e](#). Caleb took State - [commit a61d9e6](#) and Decorator. Corné ended up taking over Iterator - [commit c840c21](#) after Reneilwe hit some access-control and traversal issues that needed a design-level fix rather than just a code tweak, and he later went back into the Composite code too, sorting out some inheritance and snapshot bugs that came up once everything was being linked together. Caleb set up Docker.